

Ex. No: 9 Threading

C PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>

void *thread_function(void *arg)

{

    int i;

    for(i = 1; i <= 5; i++)

    {

        printf("Thread Executing : %d\n", i);

        sleep(1);

    }

    pthread_exit(NULL);

}

int main()

{

    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_function, NULL);

    pthread_create(&t2, NULL, thread_function, NULL);

    pthread_join(t1, NULL);

    pthread_join(t2, NULL);

    printf("All Threads Completed\n");
```

```
return 0;
}
```

Output:

```
(kali㉿kali)-[~]
$ nano Threading.c

(kali㉿kali)-[~]
$ gcc Threading.c -o Threading

(kali㉿kali)-[~]
$ ./Threading
Thread Executing : 1
Thread Executing : 1
Thread Executing : 2
Thread Executing : 2
Thread Executing : 3
Thread Executing : 3
Thread Executing : 4
Thread Executing : 4
Thread Executing : 5
Thread Executing : 5
All Threads Completed
```

SHELL SCRIPT

Shell scripting does not support threads directly like C. However, concurrent execution can be simulated using background processes (&).

```
#!/bin/bash

task1()
{
for i in 1 2 3 4 5
do
echo "Thread 1 : $i"
sleep 1
done
}
```

```
task2()
{
for i in 1 2 3 4 5
do
echo "Thread 2 : $i"
sleep 1
done
}

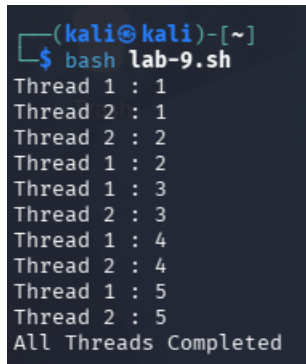
task1 &

task2 &

wait

echo "All Threads Completed"
```

Output:

A terminal window with a dark background and light blue text. The prompt is '(kali@kali)-[~]'. The user has entered '\$ bash lab-9.sh'. The output shows interleaved lines for Thread 1 and Thread 2, with Thread 2's output lagging behind Thread 1's. The final line is 'All Threads Completed'.

```
(kali@kali)-[~]
$ bash lab-9.sh
Thread 1 : 1
Thread 2 : 1
Thread 2 : 2
Thread 1 : 2
Thread 1 : 3
Thread 2 : 3
Thread 1 : 4
Thread 2 : 4
Thread 1 : 5
Thread 2 : 5
All Threads Completed
```